- `RoutingMiddleware`—The routing middleware is responsible for selecting the endpoint for an incoming request. `WebApplication` adds it by default, but as discussed in chapter 4, adding it explicitly ensures that it runs after the `StaticFileMiddleware`.
- `AuthorizationMiddleware`—This middleware controls whether an endpoint is allowed to run based on the user making the request, but requires you also to configure authentication for your application. You'll learn more about authentication in chapter 23 and authorization in chapter 24.

In addition to the middleware added explicitly, `WebApplication` automatically adds some extra middleware (as discussed in chapter 4), such as the `EndpointMiddleware`, which is automatically added to the *end* of the middleware pipeline. As with minimal APIs, the `RoutingMiddleware` selects which endpoint handler to execute, and the `EndpointMiddleware` executes the handler to generate a response.

Together, this pair of middleware is responsible for interpreting the request to determine which Razor Page to invoke, for reading parameters from the request, and for generating the final HTML. Little configuration is required; you need only add the middleware to the pipeline and specify that you want to use Razor Page endpoints by calling `MapRazorPages`. For each request, the routing middleware uses the request's URL to determine which Razor Page to invoke. Then the endpoint middleware executes the Razor Page to generate the HTML response.

When the application is configured, it can start handling requests. But *how* does it handle them? In section 13.1.3 you'll get a glimpse at Razor Pages